

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 7

Three Small Programs: Processes, Redirection, a Pipe

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-243
Daniel Nicorici

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

Purpose of the work **2**

1 Part 1. fork + exec + wait **2**

2 Part 2. A zombie, on purpose **3**

3 Part 3. Redirection: ls -l > out.txt **4**

4 Part 4. A pipe: ls /etc | wc -l **5**

Conclusions **7**

Purpose of the work

The lecture asked three questions: how a process is born, where `printf` writes, and how `ls | wc -l` works. The purpose of this lab is to run the answers: three short C programs that do by hand what the shell does for a command, for a redirection and for a pipe. Each program is compiled, run, changed in one line and run again to see what happens.

The work was done on my Arch Linux laptop with gcc 16.2.1, in an interactive bash in the folder `~/os-lab7`.

```
$ mkdir -p ~/os-lab7 && cd ~/os-lab7
$ gcc --version | head -1
gcc (GCC) 16.2.1 20260810
```

1 Part 1. fork + exec + wait

`run.c` performs the three steps of the shell for the command `ls -l`: the process copies itself with `fork`, the copy becomes `ls` with `execlp`, and the original waits for it with `waitpid`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Figure 1.1 – run.c

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 98814
child:  my PID is 98815, my parent is 98814
total 36
-rw-r--r-- 1 Nico Nico  1013 Oct 22 20:28 pipeline.c
```

```
-rw-r--r-- 1 Nico Nico   703 Oct 22 20:28 redirect.c
-rwxr-xr-x 1 Nico Nico 16472 Oct 22 20:29 run
-rw-r--r-- 1 Nico Nico   911 Oct 22 20:28 run.c
-rw-r--r-- 1 Nico Nico   979 Oct 22 20:28 zombie.c
parent: child 98815 ended with status 0
```

Figure 1.2 – run: the child becomes ls -l

Then the `execlp` line was replaced, first with `execlp("date", "date", (char *)NULL);` and then with `execlp("nosuchprog", "nosuchprog", (char *)NULL);`:

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 99360
child:  my PID is 99361, my parent is 99360
Thu Oct 22 20:29:21 EEST 2026
parent: child 99361 ended with status 0
```

Figure 1.3 – The child becomes date

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 99460
child:  my PID is 99461, my parent is 99460
exec: No such file or directory
parent: child 99461 ended with status 127
```

Figure 1.4 – A program that does not exist

Observations:

1. **Why `perror("exec")` appears only with `nosuchprog`.** A successful `execlp` never returns: it replaces the whole program of the child with `ls` or `date`, so the lines after it no longer exist in that process. `execlp` returns only when it fails, and that happened only with `nosuchprog`, which is not found in any PATH directory; then `perror` printed the reason, “No such file or directory”.
2. **The child’s parent.** The child prints `my parent is 98814`, which is the PID that the parent printed for itself on the first line: `getppid()` returns the PID of the process that called `fork`.
3. **The status.** With `ls` (and `date`) the parent sees status 0, because the program ended successfully. With `nosuchprog` it sees 127: the exec failed, so the child ran its own `exit(127)`, and `WEXITSTATUS` extracted that number.

2 Part 2. A zombie, on purpose

`zombie.c` is `run.c` (the `ls` version) with one new line before `waitpid`, so the parent sleeps for 20 seconds before it collects its child:

```
    int status;
    sleep(20);                                  /* the new line */
    waitpid(pid, &status, 0);                   /* step 3: wait for the child */
```

Figure 2.1 – The change in zombie.c

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
[1] 99669
$ sleep 2; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  98636   98633 SNs  bash -i
  99669   98636 SN   ./zombie
  99670   99669 ZN   [ls] <defunct>
  99918   98636 RN+  ps -o pid,ppid,stat,cmd
```

Figure 2.2 – The process table while the parent sleeps

```
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  98636   98633 SNs  bash -i
 100897   98636 RN+  ps -o pid,ppid,stat,cmd
[1]+  Done                    ./zombie > /dev/null
```

Figure 2.3 – The process table after 20 seconds

Observations:

1. **The STAT letter of `ls`.** `ls` has `Z`: it is a zombie, shown as `[ls] <defunct>`. The child has already finished, but its parent (PID 99669, `./zombie`) has not collected its exit status yet, so the kernel keeps a small entry with the PID and the status in the process table. The `N` next to the letters only means that the processes of my session run with a lowered priority (a positive nice value).
2. **After 20 seconds.** The `[ls] <defunct>` line is gone, and so is `./zombie`, which bash reports as `Done`. The line that removed the zombie is `waitpid(pid, &status, 0);`: when the parent finally waited, it received the exit status and the kernel freed the zombie's entry.

3 Part 3. Redirection: ls -l > out.txt

`redirect.c` does what the shell does for `ls -l > out.txt`: in the child it closes descriptor 1, opens the file, which receives the smallest free number, and then runs `ls`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                               /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always */
        perror("exec");
        exit(127);
```

```
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Figure 3.1 – redirect.c

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 76
-rw-r--r-- 1 Nico Nico     0 Oct 22 20:29 out.txt
-rw-r--r-- 1 Nico Nico  1013 Oct 22 20:28 pipeline.c
```

Figure 3.2 – redirect and the content of out.txt

Observations:

1. **The number `open` gave.** It gave descriptor 1. `open` always returns the smallest descriptor that is not in use: 0 is standard input, 1 had just been closed, and 2 is standard error, so 1 was the first free number.
2. **Why the listing is not on the screen.** `ls` was not changed: it writes to descriptor 1, as every program does. But in this child, descriptor 1 is now `out.txt`, and `exec` replaces the program while keeping the open descriptors, so `ls` writes into the file without knowing it. The message about descriptor 1 still appeared because it was written to descriptor 2, which was not touched.
3. **Redirecting the errors.** For `2> err.txt`, I would close descriptor 2 before `open`; the file would then become descriptor 2, standard error.

4 Part 4. A pipe: ls /etc | wc -l

`pipeline.c` builds the pipeline by hand with one pipe and two children. In `ls`, descriptor 1 becomes the write end `p[1]`; in `wc`, descriptor 0 becomes the read end `p[0]`. `dup` copies a descriptor into the smallest free number, just like `open`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /etc | wc -l   */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                          /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                              /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
```

```
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                          /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                              /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                                /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Figure 4.1 – pipeline.c

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
181
$ ls /etc | wc -l
181
```

Figure 4.2 – pipeline compared with the real shell

Then the parent's `close(p[1]);` was turned into a comment:

```
    close(p[0]);                                /* the parent uses neither end */
    /* close(p[1]); */
```

Figure 4.3 – The change that breaks the pipeline

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
^C
$ ./pipeline &
[1] 101527
$ sleep 1; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  98636   98633 SNs  bash -i
 101527   98636 SN   ./pipeline
 101529  101527 SN   wc -l
 101603   98636 RN+  ps -o pid,ppid,stat,cmd
$ kill %1
[1]+  Terminated                ./pipeline
```

Figure 4.4 – The broken pipeline never ends

The program printed nothing and had to be stopped with Ctrl-C. Running it once more in the background showed which processes were still alive; after that, `pipeline.c` was restored to the

original version.

Observations:

1. **The same number.** Yes: `./pipeline` and `ls /etc | wc -l` both printed 181.
2. **Which child never ends.** `wc`. In the process table there is still `wc -l` (PID 101529, child of `./pipeline`), while `ls` has already finished and been collected by the parent's first `wait`.
3. **What `wc` is waiting for.** `wc` keeps reading until end of file, and a pipe reports end of file only when every write end is closed, but the parent still holds `p[1]` open, so `wc` waits forever and the parent's second `wait` waits for `wc`.

The optional three-stage `pipeline3.c` was not done.

Conclusions

The three programs showed that the shell's everyday features are built from a few system calls. Running a command is `fork`, `exec` and `wait`: the child is a copy of the parent until `exec` replaces its program completely, which is why the code after a successful `exec` never runs and why the exit status is the only thing the parent learns. A child that has ended but has not been waited for stays in the process table as a zombie until the parent calls `wait`.

Redirection and pipes rely on one rule: a new descriptor always gets the smallest free number. Closing descriptor 1 and opening a file makes the file standard output, and `exec` keeps it, so an unchanged `ls` writes into the file. A pipe connects the standard output of one child to the standard input of another with `dup`, and it works only if every unused end is closed: a single forgotten write end in the parent was enough to make `wc` wait forever for an end of file that never came.